AGENCY PLATFORM · CONCEPT EXPLAINED

Architecture Decision Records

What an ADR is, why this project uses them, two worked examples from the repository, how to change a decision, and when to write one.

IN ONE SENTENCE

An **ADR** is a short file that records **one important technical decision**: what was decided, why, and what alternatives were rejected.

1. Why ADRs exist

Code shows *what* the system does. It never shows *why* it was built that way. Six months from now, someone (the client, a new developer, Claude Code, or you) will look at the project and ask:

> "Why is this one big application instead of microservices? That seems old-fashioned."

Without an ADR, nobody remembers the reason, and the team either re-argues the whole decision or quietly undoes it and breaks something. With an ADR, the answer is a two-minute read.

ADRs also help Claude Code here. `CLAUDE.md` says: "*Do not contradict an ADR; propose a new ADR instead.*" Every session therefore respects your earlier decisions, even though the AI has no memory between conversations. **The repository is the memory.**

2. Anatomy of an ADR: the one already in the repository

`docs/adr/0001-modular-monolith-clean-architecture.md`, annotated:

```
# ADR 0001: Modular monolith with Clean Architecture      ← number + short title (the decision itself)

Status: Accepted · Date: 2026-10-11                       ← is it in force? when was it decided?

## Decision                                               ← WHAT we chose, stated plainly
One solution and one database, split into modules (Catalogue, Contracts, Pricing, Bookings,
Partners, Billing, Treasury, GuestAssistant, Identity) inside Clean Architecture layers.
Each host (back office, partner API, public site, worker, MCP server, migrator) is its
own container and Kubernetes deployment.

## Why                                                    ← the REASONS, tied to this project's situation
Small team; business rules are tightly connected (pricing, bookings, billing);
microservices would add network, data-consistency and deployment cost with no benefit now.

## Rejected alternatives                                  ← WHAT ELSE we considered, and why not
```

```
Microservices (too costly for one developer); classic layered app without
module borders (old system's problem).
```

Section	What it answers	Tip
Number + title	Which decision is this?	The title states the decision, not the topic: "Modular monolith…", not "Architecture".
Status + date	Is it still in force? Since when?	Accepted, Superseded by 00xx, or Proposed (still under discussion).
Decision	What exactly did we choose?	One short paragraph, concrete enough to check the code against.
Why	Which facts about *this* project led here?	Write the context (team size, constraints). The context is what changes later.
Rejected alternatives	What else was considered, and why not?	The most valuable part. It shows the decision was a real choice.

WHY "REJECTED ALTERNATIVES" MATTERS MOST

If a colleague later says "let's use microservices", ADR 0001 already answers: *considered, rejected because you are one developer*. If that reason stops being true (say there are ten developers in two years), it is time for a **new** ADR. The old reasoning is still there to compare against.

3. A second example: one you will write soon

In task 3 of the walking skeleton you choose how database IDs are generated. That is a real choice with alternatives, so it gets an ADR:

```
# ADR 0007: GUID v7 primary keys

Status: Accepted · Date: 2026-10-20

## Decision
Entities use Guid v7 identifiers generated in the application (Guid.CreateVersion7()),
stored as PostgreSQL uuid.

## Why
- The ID exists before the row is saved, so a use case can return it or put it in an event
immediately.
- Partners cannot guess or count IDs (with 1, 2, 3… a partner could see how many bookings we have).
- v7 IDs are ordered by time, so inserts stay index-friendly (random v4 GUIDs fragment indexes).

## Rejected alternatives
- bigint identity columns: expose counts, need a database round trip to get the ID.
- Guid v4: random order, slower inserts on large tables.
```

Notice the pattern. Every "why" bullet is tied to something concrete in *this* system (use cases, partners, large tables). It is not a generic "best practice".

4. Changing your mind: ADRs are never edited

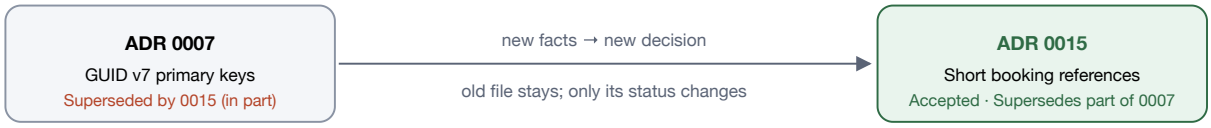
ADRs are a history log, like accounting entries. You do not rewrite them; you **supersede** them. Suppose that in phase 5 you discover partners need short, human-readable booking references:

```
# ADR 0015: Short booking references for partners
Status: Accepted · Supersedes part of 0007
...
```

Then you change only the status line of ADR 0007:

```
Status: Superseded by 0015 (booking references only)
```

Anyone reading 0007 now sees it was replaced and follows the link to 0015. This works the same way as the project's billing rule, "*issued invoices are immutable; corrections go through credit notes*". It is the same idea applied to decisions.



5. When to write one, and when not

✅ Write an ADR	❌ Don't bother
Choosing between real alternatives that are costly to reverse: database, identity provider, UI library, hosting	Code style, naming, formatting (that belongs in `.editorconfig`)
Adding a dependency that shapes the code (a test library, an API-client generator)	Fixing a bug
A security approach (where browser tokens live, BFF vs SPA)	Choices with only one sensible option
Breaking a convention on purpose ("we use MediatR after all")	Temporary experiments

> **THE SIX-MONTH TEST**
>
> Ask: "**Would someone reasonably ask 'why did we do it this way?' in six months?**" If yes, write an ADR.

6. Habits that keep ADRs useful

- **Keep each to one screen.** Nobody reads a three-page ADR.
- **Write it in the same pull request** as the code that implements the decision, so the decision and the code arrive together.
- **Write it when you decide, not weeks later.** By then you will have forgotten the rejected options.
- **Number them in order** (`0007-…`, `0008-…`) and never reuse a number.
- **Ask Claude Code to draft it** right after a decision in a session ("write an ADR for what we just decided"), then edit it so the reasons are yours.

7. Template to copy

```
docs/adr/00NN-short-decision-title.md

# ADR 00NN: <the decision, stated as a title>

Status: Accepted · Date: YYYY-MM-DD

## Decision
<one short paragraph: what we chose, concrete enough to check the code against>

## Why
<the facts about this project that led here: team, constraints, requirements>

## Rejected alternatives
<option> (<why not>); <option> (<why not>).
```

8. In this project right now

ADR	Decision
0001	Modular monolith with Clean Architecture
0002	PostgreSQL as the only database
0003	React for back office and partner portal, Next.js for the public site
0004	Managed Kubernetes with managed data services
0005	Start with an empty system (no data migration)
0006	AI drafts, humans approve

The next ones will come from the open decisions in the Project Handbook (section C3). The ✔ column there marks which ones need an ADR: GUID v7 keys, case- and accent-insensitive names, the identity provider, where browser tokens live, the API-client generator, the UI library, the ingress/gateway controller, and the cloud provider.